JavaScript po ES6

ES6/ES2015

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

02.10.2026

CC BY-NC-SA 4.0

JavaScript po ES6

Úvod

V této přednášce se podíváme na **ES6 (ES2015)** – revoluci, která definovala moderní JavaScript

Úvod

V této přednášce se podíváme na **ES6 (ES2015)** – revoluci, která definovala moderní JavaScript

Probereme si:

Úvod

V této přednášce se podíváme na **ES6 (ES2015)** – revoluci, která definovala moderní JavaScript

Probereme si:

- Nové funkce a vylepšení jazyka

Úvod

V této přednášce se podíváme na **ES6 (ES2015)** – revoluci, která definovala moderní JavaScript

Probereme si:

- Nové funkce a vylepšení jazyka
- Jak se změnil způsob psaní kódu

Úvod

V této přednášce se podíváme na **ES6 (ES2015)** – revoluci, která definovala moderní JavaScript

Probereme si:

- Nové funkce a vylepšení jazyka
- Jak se změnil způsob psaní kódu
- Jak se zjednodušilo asynchronní programování

Úvod

V této přednášce se podíváme na **ES6 (ES2015)** – revoluci, která definovala moderní JavaScript

Probereme si:

- Nové funkce a vylepšení jazyka
- Jak se změnil způsob psaní kódu
- Jak se zjednodušilo asynchronní programování
- Jak se zlepšila práce s objekty a třídami

Úvod

V této přednášce se podíváme na **ES6 (ES2015)** – revoluci, která definovala moderní JavaScript

Probereme si:

- Nové funkce a vylepšení jazyka
- Jak se změnil způsob psaní kódu
- Jak se zjednodušilo asynchronní programování
- Jak se zlepšila práce s objekty a třídami

Zaměříme se především na praktické použití nových funkcí

Úvod

Proč je důležité znát nové verze JavaScriptu?

Úvod

Proč je důležité znát nové verze JavaScriptu?

- Modernější a čitelnější kód

Úvod

Proč je důležité znát nové verze JavaScriptu?

- Modernější a čitelnější kód
- Lepší výkon a efektivita

Úvod

Proč je důležité znát nové verze JavaScriptu?

- Modernější a čitelnější kód
- Lepší výkon a efektivita
- Nové možnosti a funkce

Úvod

Proč je důležité znát nové verze JavaScriptu?

- Modernější a čitelnější kód
- Lepší výkon a efektivita
- Nové možnosti a funkce
- Většina moderních frameworků využívá nové funkce

ES6/ES2015

let a const

ES6 přináší nové způsoby deklarace proměnných:

let a const

ES6 přináší nové způsoby deklarace proměnných: `let` a `const`

let a const

ES6 přináší nové způsoby deklarace proměnných: `let` a `const`

```
let x = 10;         // variable that can be changed
const y = 20;       // constant that cannot be changed
```

let a const

ES6 přináší nové způsoby deklarace proměnných: `let` a `const`

```
let x = 10;         // variable that can be changed
const y = 20;       // constant that cannot be changed
```

Hlavní rozdíly oproti `var`:

let a const

ES6 přináší nové způsoby deklarace proměnných: `let` a `const`

```
let x = 10;         // variable that can be changed
const y = 20;       // constant that cannot be changed
```

Hlavní rozdíly oproti `var`:

- Block scope místo function scope

let a const

ES6 přináší nové způsoby deklarace proměnných: `let` a `const`

```
let x = 10;         // variable that can be changed
const y = 20;       // constant that cannot be changed
```

Hlavní rozdíly oproti `var`:

- Block scope místo function scope
- Nelze redefinovat ve stejném scope

let a const

ES6 přináší nové způsoby deklarace proměnných: `let` a `const`

```
let x = 10;         // variable that can be changed
const y = 20;       // constant that cannot be changed
```

Hlavní rozdíly oproti `var`:

- Block scope místo function scope
- Nelze redefinovat ve stejném scope
- Temporal Dead Zone

let a const

ES6 přináší nové způsoby deklarace proměnných: `let` a `const`

```
let x = 10;         // variable that can be changed
const y = 20;       // constant that cannot be changed
```

Hlavní rozdíly oproti `var`:

- Block scope místo function scope
- Nelze redefinovat ve stejném scope
- Temporal Dead Zone
- Přísnější pravidla pro použití

let a const

Příklad problému s `var`:

```
var x = 10;
var x = 20; // OK - can be redefined

function example() {
  console.log(x); // undefined - hoisting
  var x = 30;
}
example();
```

let a const

```
let x = 10;
let x = 20; // Error: Identifier 'x' has already been
declared
```

let a const

```
let x = 10;

function example() {
  console.log(x); // ReferenceError: Cannot access 'x' before
initialization - Temporal Dead Zone
  let x = 30;
}
example();
```

let a const

`const` zabraňuje

let a const

`const` zabraňuje změně hodnoty u primitivních datových typů a reference u referenčních datových typů

```
const PI = 3.14159;
PI = 3; // TypeError: Assignment to constant variable
```

let a const

Ale pozor na objekty a pole:

let a const

Ale pozor na objekty a pole:

```
const user = { name: "John" };
user.name = "Jane"; // OK - changing property, not reference
console.log(user); // { name: "Jane" }
user = { name: "Mike" }; // TypeError: Assignment to constant
variable.
```

let a const

Ale pozor na objekty a pole:

```
const user = { name: "John" };
user.name = "Jane"; // OK - changing property, not reference
console.log(user); // { name: "Jane" }
user = { name: "Mike" }; // TypeError: Assignment to constant
variable.
```

```
const arr = [1, 2, 3];
arr.push(4); // OK - modifying array content
console.log(arr); // [1, 2, 3, 4]
arr = [5, 6]; // TypeError: Assignment to constant variable.
```

Block scope a Temporal Dead Zone

Blok kódu je

Block scope a Temporal Dead Zone

Blok kódu je sekvence příkazů uzavřená mezi `{` a `}`

Block scope a Temporal Dead Zone

Blok kódu je sekvence příkazů uzavřená mezi `{` a `}`

Block scope je

Block scope a Temporal Dead Zone

Blok kódu je sekvence příkazů uzavřená mezi `{` a `}`

Block scope je typ rozsahu platnosti proměnných, kde jsou proměnné dostupné pouze v bloku, kde byly deklarovány

Block scope a Temporal Dead Zone

Blok kódu je sekvence příkazů uzavřená mezi `{` a `}`

Block scope je typ rozsahu platnosti proměnných, kde jsou proměnné dostupné pouze v bloku, kde byly deklarovány

```
if (true) {
  var x = 10; // function scope
  let y = 20; // block scope
}
console.log(x); // 10
console.log(y); // ReferenceError: y is not defined
```

Block scope a Temporal Dead Zone

To samé platí pro cykly:

```
for (var i = 0; i < 3; i++) {
  // i is available in the whole function scope
}
console.log(i); // 3
```

Block scope a Temporal Dead Zone

To samé platí pro cykly:

```
for (var i = 0; i < 3; i++) {
  // i is available in the whole function scope
}
console.log(i); // 3

for (let j = 0; j < 3; j++) {
  // j is only available in this block scope
}
console.log(j); // ReferenceError: j is not defined
```

Block scope a Temporal Dead Zone

Temporal Dead Zone (TDZ) proměnné je

Block scope a Temporal Dead Zone

Temporal Dead Zone (TDZ) proměnné je sekce kódu mezi vstupem do scope a deklarací této proměnné

Block scope a Temporal Dead Zone

Temporal Dead Zone (TDZ) proměnné je sekce kódu mezi vstupem do scope a deklarací této proměnné

V této době JavaScript ví o proměnné (hoisting), ale zabraňuje jejímu použití v této sekci

Block scope a Temporal Dead Zone

Temporal Dead Zone (TDZ) proměnné je sekce kódu mezi vstupem do scope a deklarací této proměnné

V této době JavaScript ví o proměnné (hoisting), ale zabraňuje jejímu použití v této sekci

```
console.log(x); // undefined - hoisting with var
var x = 1;

console.log(y); // ReferenceError - TDZ with let
let y = 2;
```

Block scope a Temporal Dead Zone

TDZ chrání před použitím proměnných před jejich deklarací:

```
const x = y; // ReferenceError - y is in TDZ
const y = 1;
```

Arrow funkce

Arrow funkce jsou

Arrow funkce

Arrow funkce jsou nový způsob zápisu funkcí s kratší syntaxí

Arrow funkce

Arrow funkce jsou nový způsob zápisu funkcí s kratší syntaxí

```
// Classic function
function add(a, b) {
  return a + b;
}

// Arrow function
const add = (a, b) => {
  return a + b;
}
```

Arrow funkce

```
// Shorthand for simple expressions
const add = (a, b) => a + b;
```

Arrow funkce

Arrow funkce mají několik **speciálních vlastností**:

Arrow funkce

Arrow funkce mají několik **speciálních vlastností**:

1. Nemají vlastní `this` - používají `this` z

Arrow funkce

Arrow funkce mají několik **speciálních vlastností**:

1. Nemají vlastní `this` - používají `this` z okolního scope

Arrow funkce

Arrow funkce mají několik **speciálních vlastností**:

1. Nemají vlastní `this` - používají `this` z okolního scope
2. Nemají vlastní `arguments` objekt

Arrow funkce

Arrow funkce mají několik **speciálních vlastností**:

1. Nemají vlastní `this` - používají `this` z okolního scope
2. Nemají vlastní `arguments` objekt
3. Nemohou být použity jako konstruktory

Arrow funkce

Arrow funkce mají několik **speciálních vlastností**:

1. Nemají vlastní `this` - používají `this` z okolního scope
2. Nemají vlastní `arguments` objekt
3. Nemohou být použity jako konstruktory
4. Neměly by se používat jako metody objektů (kvůli `this`)

Arrow funkce

Arrow funkce mají několik **speciálních vlastností**:

1. Nemají vlastní `this` - používají `this` z okolního scope
2. Nemají vlastní `arguments` objekt
3. Nemohou být použity jako konstruktory
4. Neměly by se používat jako metody objektů (kvůli `this`)
5. Zkrácený zápis funguje pouze pro jednoduché výrazy

Arrow funkce

Řešení problému s `this` pomocí arrow funkcí:

```
function Person(name) {
  this.name = name;

  // Problem with this in classic function
  this.sayHiLater = function() {
    setTimeout(function() {
      // this is not Person
      console.log('Hello, ' + this.name);
    }, 1000);
  };
```

Arrow funkce

```
  // Solution with arrow function
  this.sayHiLaterArrow = function() {
    setTimeout(() => {
      // this is Person
      console.log('Hello, ' + this.name);
    }, 1000);
  };
}
```

Template literals

Template literals jsou

Template literals

Template literals jsou nový způsob zápisu řetězců pomocí

Template literals

Template literals jsou nový způsob zápisu řetězců pomocí zpětných apostrofů (backticks)

Template literals

Template literals jsou nový způsob zápisu řetězců pomocí zpětných apostrofů (backticks)

```
// Classic way
var name = "John";
var greeting = "Ahoj " + name + "!";

// Template literal
const name = "John";
const greeting = `Ahoj ${name}!`;
```

Template literals

Výhody:

- Interpolace výrazů pomocí `${}`

Template literals

Výhody:

- Interpolace výrazů pomocí `${}`
- Víceřádkové řetězce

Template literals

Výhody:

- Interpolace výrazů pomocí `${}`
- Víceřádkové řetězce
- Možnost vnořených výrazů

Template literals

Víceřádkové řetězce bez escapování:

```
// Old way
var html = "<div>\n" +
  "  <h1>Title</h1>\n" +
  "  <p>Paragraph</p>\n" +
  "</div>";
```

Template literals

```
// Template literal
const html = `<div>
  <h1>Title</h1>
  <p>Paragraph</p>
</div>`;
```

Template literals

Interpolace složitějších výrazů:

```
const x = 10;
const y = 20;
console.log(`${x} + ${y} = ${x + y}`);
// "10 + 20 = 30"
```

Template literals

Interpolace složitějších výrazů:

```
const x = 10;
const y = 20;
console.log(`${x} + ${y} = ${x + y}`);
// "10 + 20 = 30"

const items = ["a", "b", "c"];
console.log(`Items: ${items.join(", ")}`);
// "Items: a, b, c"
```

Template literals

```
const person = { name: "John", age: 30 };
console.log(
  `${person.name} is ${person.age > 18 ? "adult" : "minor"}`
);
// "John is adult"
```

Destructuring

Destructuring umožňuje

Destructuring

Destructuring umožňuje rozbalit hodnoty z polí nebo vlastnosti z objektů

Destructuring

Destructuring umožňuje rozbalit hodnoty z polí nebo vlastnosti z objektů do samostatných proměnných

Destructuring

Destructuring umožňuje rozbalit hodnoty z polí nebo vlastnosti z objektů do samostatných proměnných

```
// Array destructuring
const numbers = [1, 2, 3];
const [first, second, third] = numbers;

console.log(first);  // 1
console.log(second); // 2
console.log(third);  // 3
```

Destructuring

```
// Object destructuring
const person = { name: "John", age: 30 };
const { name, age } = person;

console.log(name); // "John"
console.log(age);  // 30
```

Destructuring

Pokročilé možnosti destructuringu:

```
// Variable renaming
const { name: firstName, age: years } = person;
console.log(firstName); // "John"
console.log(years);     // 30

// Default values
const { name, age, country = "USA" } = person;
console.log(country); // "USA"
```

Destructuring

```
// Nested destructuring
const user = {
  id: 1,
  info: { firstName: "John", lastName: "Doe" }
};
const { info: { firstName, lastName } } = user;

console.log(firstName); // "John"
console.log(lastName);  // "Doe"
```

Destructuring

Pokud chcete destrukturovat celý podobjekt a zároveň některé jeho konkrétní vlastnosti, musíte

Destructuring

Pokud chcete destrukturovat celý podobjekt a zároveň některé jeho konkrétní vlastnosti, musíte specifikovat oba výrazy

```
const { info, info: { firstName, lastName } } = user;

console.log(info); // { firstName: "John", lastName: "Doe" }
console.log(firstName); // "John"
console.log(lastName);  // "Doe"
```

Destructuring

Destructuring v parametrech funkcí:

```
// Old way
function printUser(user) {
  console.log(user.name + " is " + user.age + " years old");
}
```

Destructuring

```
// With destructuring
function printUser({ name, age }) {
  console.log(`${name} is ${age} years old`);
}

// Call remains the same
printUser({ name: "John", age: 30 });
```

Destructuring

```
// With destructuring
function printUser({ name, age }) {
  console.log(`${name} is ${age} years old`);
}

// Call remains the same
printUser({ name: "John", age: 30 });
```

Výhodou přijímání parametrů pomocí jednoho objektu a pomocí destructuringu je, že

Destructuring

```
// With destructuring
function printUser({ name, age }) {
  console.log(`${name} is ${age} years old`);
}

// Call remains the same
printUser({ name: "John", age: 30 });
```

Výhodou přijímání parametrů pomocí jednoho objektu a pomocí destructuringu je, že nezáleží na pořadí parametrů

Default, rest a spread operátory

Defaultní hodnoty parametrů umožňují

Default, rest a spread operátory

Defaultní hodnoty parametrů umožňují definovat výchozí hodnoty parametrů

Default, rest a spread operátory

Defaultní hodnoty parametrů umožňují definovat výchozí hodnoty parametrů

```
// Old way
function greet(name) {
  name = name || "Guest";
  return "Hello, " + name;
}
```

Default, rest a spread operátory

```
// With default parameters
function greet(name = "Guest") {
  return `Hello, ${name}`;
}

console.log(greet()); // "Hello, Guest"
console.log(greet("John")); // "Hello, John"
```

Pomocné funkce pro pole

Ještě před ES6 byly na polích některé užitečné funkce, které si ukážeme

Pomocné funkce pro pole

Ještě před ES6 byly na polích některé užitečné funkce, které si ukážeme

- **map** - transformuje prvky pole (např. vynásobí každý prvek číslem 2)

Pomocné funkce pro pole

Ještě před ES6 byly na polích některé užitečné funkce, které si ukážeme

- **map** - transformuje prvky pole (např. vynásobí každý prvek číslem 2)
- **filter** - filtruje prvky pole (vrací pouze prvky, které splňují podmínku)

Pomocné funkce pro pole

Ještě před ES6 byly na polích některé užitečné funkce, které si ukážeme

- **map** - transformuje prvky pole (např. vynásobí každý prvek číslem 2)
- **filter** - filtruje prvky pole (vrací pouze prvky, které splňují podmínku)
- **reduce** - redukuje pole na jednu hodnotu (např. součet prvků)

Pomocné funkce pro pole

Ještě před ES6 byly na polích některé užitečné funkce, které si ukážeme

- **map** - transformuje prvky pole (např. vynásobí každý prvek číslem 2)
- **filter** - filtruje prvky pole (vrací pouze prvky, které splňují podmínku)
- **reduce** - redukuje pole na jednu hodnotu (např. součet prvků)
- **some** - testuje, zda alespoň jeden prvek pole splňuje podmínku

Pomocné funkce pro pole

Ještě před ES6 byly na polích některé užitečné funkce, které si ukážeme

- **map** - transformuje prvky pole (např. vynásobí každý prvek číslem 2)
- **filter** - filtruje prvky pole (vrací pouze prvky, které splňují podmínku)
- **reduce** - redukuje pole na jednu hodnotu (např. součet prvků)
- **some** - testuje, zda alespoň jeden prvek pole splňuje podmínku
- **every** - testuje, zda všechny prvky pole splňují podmínku

Funkce map

Funkce map umožňuje

Funkce map

Funkce map umožňuje transformovat prvky pole pomocí transformační funkce (předaná jako parametr) a vrátit nové pole s transformovanými prvky

Funkce map

Funkce map umožňuje transformovat prvky pole pomocí transformační funkce (předaná jako parametr) a vrátit nové pole s transformovanými prvky

```
const numbers = [1, 2, 3];
const doubled = numbers.map(x => x * 2);
console.log(doubled); // [2, 4, 6]
```

Funkce filter

Funkce filter umožňuje

Funkce filter

Funkce filter umožňuje filtrovat prvky pole pomocí filtrační funkce (předané jako parametr)

Funkce filter

Funkce filter umožňuje filtrovat prvky pole pomocí filtrační funkce (předané jako parametr)

```
const numbers = [1, 2, 3];
const even = numbers.filter(x => x % 2 === 0);
console.log(even); // [2]
```

Funkce reduce

Funkce reduce umožňuje

Funkce reduce

Funkce reduce umožňuje redukovat pole na jednu hodnotu pomocí redukční funkce (předané jako parametr)

Funkce reduce

Funkce reduce umožňuje redukovat pole na jednu hodnotu pomocí redukční funkce (předané jako parametr)

Funkce `reduce` má dva parametry:

Funkce reduce

Funkce reduce umožňuje redukovat pole na jednu hodnotu pomocí redukční funkce (předané jako parametr)

Funkce `reduce` má dva parametry:

- Funkce, která se volá pro každý prvek pole
- Počáteční hodnota

Funkce přijímá dva parametry - akumulovanou hodnotu a aktuální prvek

Funkce reduce

Funkce reduce umožňuje redukovat pole na jednu hodnotu pomocí redukční funkce (předané jako parametr)

Funkce `reduce` má dva parametry:

- Funkce, která se volá pro každý prvek pole
- Počáteční hodnota

Funkce přijímá dva parametry - akumulovanou hodnotu a aktuální prvek

Výsledek funkce je

Funkce reduce

Funkce reduce umožňuje redukovat pole na jednu hodnotu pomocí redukční funkce (předané jako parametr)

Funkce `reduce` má dva parametry:

- Funkce, která se volá pro každý prvek pole
- Počáteční hodnota

Funkce přijímá dva parametry - akumulovanou hodnotu a aktuální prvek

Výsledek funkce je akumulační argument pro další volání funkce

Funkce reduce

```
const numbers = [1, 2, 3, 4];
const sum = numbers.reduce(
  (total, n) => total + n,
  0
);
console.log(sum); // 10
```

Funkce some

Funkce some umožňuje

Funkce some

Funkce some umožňuje testovat, zda alespoň jeden prvek pole splňuje podmínku

```
const numbers = [1, 2, 3, 4];
const hasEven = numbers.some(x => x % 2 === 0);
console.log(hasEven); // true
```

Funkce every

Funkce every umožňuje

Funkce every

Funkce every umožňuje testovat, zda všechny prvky pole splňují podmínku

```
const numbers = [1, 2, 3, 4];
const allEven = numbers.every(x => x % 2 === 0);
console.log(allEven); // false
```

Rest parametr

Rest parametr umožňuje

Rest parametr

Rest parametr umožňuje zachytit zbývající argumenty do pole

Rest parametr

Rest parametr umožňuje zachytit zbývající argumenty do pole

```
// Old way with arguments
function sum() {
  var numbers = Array.prototype.slice.call(arguments);
  return numbers.reduce(function(total, n) {
    return total + n;
  }, 0);
}
```

Rest parametr

```
// With rest parameter
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}

console.log(sum(1, 2, 3, 4)); // 10
```

Rest parametr

```
function sum(a, b, ...numbers) {
  console.log(a); // 1
  console.log(b); // 2
  console.log(numbers); // [3, 4]
  return numbers.reduce((total, n) => total + n, a + b);
}

console.log(sum(1, 2, 3, 4)); // 10
```

Spread operátor

Spread operátor umožňuje

Spread operátor

Spread operátor umožňuje rozložit pole nebo objekt

Spread operátor

Spread operátor umožňuje rozložit pole nebo objekt

```
// Spread for arrays
const numbers = [1, 2, 3];
console.log(Math.max(...numbers)); // 3

const arr1 = [1, 2];
const arr2 = [...arr1, 3, 4]; // [1, 2, 3, 4]
```

Spread operátor

```
// Spread for objects (ES9/ES2018+)
const defaults = { theme: "dark", lang: "en" };
const user = { ...defaults, name: "John" };
// { theme: "dark", lang: "en", name: "John" }
```

Spread operátor

```
// Spread for objects (ES9/ES2018+)
const defaults = { theme: "dark", lang: "en" };
const user = { ...defaults, name: "John" };
// { theme: "dark", lang: "en", name: "John" }

// Default overridden
const user2 = { ...defaults, theme: "light", name: "Jane" };
// { theme: "light", lang: "en", name: "Jane" }
```

Třídy

Třídy poskytují čistší syntax pro objektově orientované programování

Třídy

Třídy poskytují čistší syntax pro objektově orientované programování

```
// Old way with prototype
function Person(name) {
  this.name = name;
}

Person.prototype.sayHello = function() {
  console.log("Hello, " + this.name);
};
```

Třídy

```
// With classes
class Person {
  constructor(name) {
    this.name = name;
  }

  sayHello() {
    console.log(`Hello, ${this.name}`);
  }
}
```

Třídy

Třídy podporují **dědičnost** pomocí `extends` a `super`

```
class Employee extends Person {
  constructor(name, role) {
    super(name); // Calls parent class constructor
    this.role = role;
  }
  sayHello() {
    super.sayHello(); // Calls parent class method
    console.log(`I am ${this.role}`);
  }
}
```

Třídy

```
const emp = new Employee("John", "developer");
emp.sayHello();
// "Hello, John"
// "I am developer"
```

Třídy

Třídy mohou mít **gettery**, **settery** a **statické metody**

```
class Circle {
  constructor(radius) {
    this._radius = radius;
  }

  // Getter
  get radius() {
    return this._radius;
  }
```

Třídy

```
  // Setter
  set radius(value) {
    if (value <= 0)
      throw new Error("Radius must be positive");
    this._radius = value;
  }
  // Static method
  static createUnit() {
    return new Circle(1);
  }
}
```

Enhanced object literals

ES6 přináší vylepšení pro zápis objektových literálů:

Enhanced object literals

ES6 přináší vylepšení pro zápis objektových literálů:

1. Zkrácený zápis vlastností

Enhanced object literals

ES6 přináší vylepšení pro zápis objektových literálů:

1. Zkrácený zápis vlastností
2. Zkrácený zápis metod

Enhanced object literals

ES6 přináší vylepšení pro zápis objektových literálů:

1. Zkrácený zápis vlastností
2. Zkrácený zápis metod
3. Computed property names

Enhanced object literals

Zkrácený zápis vlastností:

```
const name = "John";
const age = 30;

// Old way
const person = {
  name: name,
  age: age
};
```

Enhanced object literals

```
// Shorthand notation
const person = { name, age };
```

Enhanced object literals

Zkrácený zápis metod:

```
// Old way
const calculator = {
  add: function(a, b) {
    return a + b;
  },
  subtract: function(a, b) {
    return a - b;
  }
};
```

Enhanced object literals

```
// Shorthand notation
const calculator = {
  add(a, b) {
    return a + b;
  },
  subtract(a, b) {
    return a - b;
  }
};
```

Computed property names

```
const prefix = "user_";

// Old way
const users = {};
users[prefix + "1"] = "John";
users[prefix + "2"] = "Jane";
```

Computed property names

```
// New way
const users = {
  [prefix + "1"]: "John",
  [prefix + "2"]: "Jane",
  [`${prefix}3`]: "Mike"
};
```

Promises

Promise je objekt reprezentující

Promises

Promise je objekt reprezentující eventuální dokončení (nebo selhání) asynchronní operace

Promises

Promise je objekt reprezentující eventuální dokončení (nebo selhání) asynchronní operace

Při konstrukci Promise se předávají dva callbacky:

Promises

Promise je objekt reprezentující eventuální dokončení (nebo selhání) asynchronní operace

Při konstrukci Promise se předávají dva callbacky:

- `resolve` - volá se, když

Promises

Promise je objekt reprezentující eventuální dokončení (nebo selhání) asynchronní operace

Při konstrukci Promise se předávají dva callbacky:

- `resolve` - volá se, když operace proběhne úspěšně
- `reject` - volá se, když

Promises

Promise je objekt reprezentující eventuální dokončení (nebo selhání) asynchronní operace

Při konstrukci Promise se předávají dva callbacky:

- `resolve` - volá se, když operace proběhne úspěšně
- `reject` - volá se, když operace selže

Promises

```
// Creating a Promise
const promise = new Promise((resolve, reject) => {
  // Asynchronous operation
  setTimeout(() => {
    const random = Math.random();
    if (random > 0.5) {
      resolve("Success!");
    } else { reject("Failed!"); }
  }, 1000);
});
```

Promises

S promise se pracuje pomocí metod `then`, `catch` a `finally`:

```
promise
  .then(result => console.log("Success:", result))
  .catch(error => console.log("Error:", error))
  .finally(() => console.log("Finished"));
```

Promises

Řetězení Promise:

```
fetchUser(1)
  .then(user => fetchUserPosts(user.id))
  .then(posts => renderPosts(posts))
  .catch(error => handleError(error));
```

Promises

Promise kombinátory:

```
// Promise.all - waits for all Promises, none can fail
Promise.all([
  fetchUser(1),
  fetchUser(2),
  fetchUser(3)
]).then(users => console.log(users));
```

Promises

```
// Promise.race - returns first completed Promise
Promise.race([
  fetch("/api/1"),
  fetch("/api/2")
]).then(response => console.log(response));
```

Moduly

ES6 přináší nativní podporu pro **moduly**

Moduly

ES6 přináší nativní podporu pro **moduly**

Module je

Moduly

ES6 přináší nativní podporu pro **moduly**

Module je **soubor s kódem**, který může obsahovat **exporty** a **importy**

Moduly

Moduly zajišťují:

Moduly

Moduly zajišťují:

- **Zapouzdření**: každý modul má svůj vlastní scope - proměnné a funkce definované v modulu nejsou automaticky dostupné v globálním scope

Moduly

Moduly zajišťují:

- **Zapouzdření**: každý modul má svůj vlastní scope - proměnné a funkce definované v modulu nejsou automaticky dostupné v globálním scope
- **Znovupoužitelnost**: kód lze organizovat do samostatných souborů a znovu používat v různých částech aplikace

Moduly

Moduly zajišťují:

- **Zapouzdření**: každý modul má svůj vlastní scope - proměnné a funkce definované v modulu nejsou automaticky dostupné v globálním scope
- **Znovupoužitelnost**: kód lze organizovat do samostatných souborů a znovu používat v různých částech aplikace
- **Správa závislostí**: moduly mohou explicitně deklarovat své závislosti pomocí importů

Moduly

```
// math.js
export const add = (a, b) => a + b;
export const subtract = (a, b) => a - b;

export default class Calculator {
  multiply(a, b) {
    return a * b;
  }
}
```

Moduly

```
// main.js
import Calculator, { add, subtract } from './math.js';

console.log(add(2, 3));         // 5
console.log(subtract(5, 2));    // 3

const calc = new Calculator();
console.log(calc.multiply(2, 3)); // 6
```

Moduly

Různé způsoby exportu:

```
// Named exports
export const name = "John";
export function sayHello() { }

// Default export (only one per module)
export default class Person { }
```

Moduly

```
// Export list
const x = 1, y = 2;
export { x, y };

// Export with renaming
export { x as numberX, y as numberY };
```

Moduly

Různé způsoby importu:

```
// Import default export
import Person from './person.js';

// Import named exports
import { name, sayHello } from './utils.js';

// Import with renaming
import { name as userName } from './user.js';
```

Moduly

```
// Import everything
import * as utils from './utils.js';
```

Výsledkem je objekt, který obsahuje všechny exportované položky

Moduly

```
// Import everything
import * as utils from './utils.js';
```

Výsledkem je objekt, který obsahuje všechny exportované položky

```
// Import for side effects
import './polyfills.js';
```

Výsledkem tohoto kódu je spuštění kódu z importovaného souboru v aktuálním kontextu

Iterátory

Iterátor je objekt, který definuje způsob průchodu sekvencí hodnot

Iterátory

Iterátor je objekt, který definuje způsob průchodu sekvencí hodnot

Výhodou iterátorů jsou

Iterátory

Iterátor je objekt, který definuje způsob průchodu sekvencí hodnot

Výhodou iterátorů jsou

- Unifikace způsobu průchodu sekvencí hodnot

Iterátory

Iterátor je objekt, který definuje způsob průchodu sekvencí hodnot

Výhodou iterátorů jsou

- Unifikace způsobu průchodu sekvencí hodnot
- **Lazy evaluation** - hodnoty se generují až když jsou potřeba

Iterátory

```
const arr = [1, 2, 3];
const iterator = arr[Symbol.iterator]();

console.log(iterator.next());
// { value: 1, done: false }
console.log(iterator.next());
// { value: 2, done: false }
console.log(iterator.next());
// { value: 3, done: false }
console.log(iterator.next());
// { value: undefined, done: true }
```

Iterátory

`Symbol.iterator` je speciální symbol, který se používá k získání iterátoru pro podporované typy

Iterátory

`Symbol.iterator` je speciální symbol, který se používá k získání iterátoru pro podporované typy

Existuje na prototype chainu

```
const arr = [1, 2, 3];
console.log(arr.__proto__)
// ...
// Symbol(Symbol.iterator): ƒ values()
// ...
```

Iterátory

Iterátor je pouze

Iterátory

Iterátor je pouze **interface** pro průchod sekvencí hodnot

Iterátory

Iterátor je pouze **interface** pro průchod sekvencí hodnot

Musí obsahovat funkci `next`, která vrací objekt s vlastnostmi

Iterátory

Iterátor je pouze **interface** pro průchod sekvencí hodnot

Musí obsahovat funkci `next`, která vrací objekt s vlastnostmi

- `value` indikující aktuální hodnotu

Iterátory

Iterátor je pouze **interface** pro průchod sekvencí hodnot

Musí obsahovat funkci `next`, která vrací objekt s vlastnostmi

- `value` indikující aktuální hodnotu
- `done` indikující, zda je iterace dokončena

Generátory

Generátor je funkce, která může být pozastavena a později znovu spuštěna tam, kde byla pozastavena

Generátory

Generátor je funkce, která může být pozastavena a později znovu spuštěna tam, kde byla pozastavena

Označení * před názvem funkce znamená, že se jedná o pozastavitelnou funkci, která automaticky vrací iterátor

```
function* numberGenerator() {
  yield 1;
  yield 2;
  yield 3;
}
```

Generátory

```
const gen = numberGenerator();

console.log(gen.next()); // { value: 1, done: false }
console.log(gen.next()); // { value: 2, done: false }
console.log(gen.next()); // { value: 3, done: false }
console.log(gen.next()); // { value: undefined, done: true }
```

Ukázka pozastavování

```
function* generator() {
  console.log("generating 1");
  yield 1;
  console.log("generating 2");
  yield 2;
  console.log("generating 3");
  yield 3;
}
```

Ukázka pozastavování

```
const gen = generator();
console.log("consuming 1", gen.next());
// generating 1
// consuming 1 { value: 1, done: false }
console.log("consuming 2", gen.next());
// generating 2
// consuming 2 { value: 2, done: false }
console.log("consuming 3", gen.next());
// generating 3
// consuming 3 { value: 3, done: false }
```

Ukázka pozastavování

```
console.log("consuming 4?", gen.next());
// consuming 4? { value: undefined, done: true }
```

Ukázka pozastavování

Praktické použití generátorů:

```
// Infinite sequence
function* fibonacci() {
  let prev = 0, curr = 1;
  while (true) {
    yield curr;
    [prev, curr] = [curr, prev + curr];
  }
}
```

Ukázka pozastavování

```
const fib = fibonacci();

console.log(fib.next().value); // 1
console.log(fib.next().value); // 1
console.log(fib.next().value); // 2
console.log(fib.next().value); // 3
console.log(fib.next().value); // 5
```

for-of cyklus

for-of cyklus umožňuje iterovat přes iterovatelné objekty

for-of cyklus

for-of cyklus umožňuje iterovat přes iterovatelné objekty

Oproti `for-in` cyklu, který iteruje přes vlastnosti objektu nebo indexy pole, `for-of` cyklus

for-of cyklus

for-of cyklus umožňuje iterovat přes iterovatelné objekty

Oproti `for-in` cyklu, který iteruje přes vlastnosti objektu nebo indexy pole, `for-of` cyklus **iteruje přes hodnoty**

for-of cyklus

for-of cyklus

```
// Array
const numbers = [1, 2, 3];
for (const num of numbers) {
  console.log(num);
}
// 1
// 2
// 3
```

for-of cyklus

```
// String
const str = "Hello";
for (const char of str) {
  console.log(char);
}
// H
// e
// l
// l
// o
```

for-of cyklus

Rozdíl mezi for-in a for-of:

```
const arr = ["a", "b", "c"];

for (const index in arr) {
  console.log(index);
}
// 0 1 2
for (const value of arr) {
  console.log(value);
}
// a b c
```

for-of cyklus

for-of funguje se všemi iterovatelními objekty, které si ukážeme později:

```
// Set
const set = new Set([1, 2, 3]);
for (const num of set) {
  console.log(num);
}
// 1 2 3
```

for-of cyklus

```
// Map
const map = new Map([["a", 1], ["b", 2]]);
for (const [key, value] of map) {
  console.log(key, value);
}
// a 1
// b 2
```

for-of cyklus

```
// Generator
function* gen() {
  yield 1;
  yield 2;
}

for (const value of gen()) {
  console.log(value);
}
// 1 2
```

Symbol

Symbol je nový primitivní datový typ pro unikátní identifikátory

Symbol

Symbol je nový primitivní datový typ pro unikátní identifikátory

```
// Each Symbol is unique
const sym1 = Symbol();
const sym2 = Symbol();
console.log(sym1 === sym2); // false

// Symbols can have description
const sym3 = Symbol("my symbol");
console.log(sym3.toString()); // "Symbol(my symbol)"
```

Symbol

Použití Symbolů jako klíčů objektů:

```
const MY_KEY = Symbol();
const obj = {
  [MY_KEY]: "secret value"
};

console.log(obj[MY_KEY]); // "secret value"
```

Symbol

```
// Symbols are not visible in regular loops
for (let key in obj) {
  console.log(key); // nothing is printed
}
console.log(Object.keys(obj)); // []
```

Symbol

Well-known Symbols:

```
// Symbol.iterator - defines iterator
const myIterable = {
  *[Symbol.iterator]() {
    yield 1;
    yield 2;
  }
};
```

Symbol

```
// Symbol.toStringTag - modifies toString() output
class MyClass {
  get [Symbol.toStringTag]() {
    return "MyClass Tag";
  }
}
console.log((new MyClass()).toString());
// [object MyClass Tag]
```

Map, Set, WeakMap, WeakSet

Map je kolekce klíč-hodnota, kde klíče mohou být libovolného typu

Map, Set, WeakMap, WeakSet

Map je kolekce klíč-hodnota, kde klíče mohou být libovolného typu

Výhody Map oproti objektu:

Map, Set, WeakMap, WeakSet

Map je kolekce klíč-hodnota, kde klíče mohou být libovolného typu

Výhody Map oproti objektu:

- Klíče mohou být libovolného typu
  - Objekty mohou mít jako klíče pouze string nebo Symbol

Map, Set, WeakMap, WeakSet

Map je kolekce klíč-hodnota, kde klíče mohou být libovolného typu

Výhody Map oproti objektu:

- Klíče mohou být libovolného typu
  - Objekty mohou mít jako klíče pouze string nebo Symbol
- Map ukládá pořadí uložených hodnot

Map, Set, WeakMap, WeakSet

Map je kolekce klíč-hodnota, kde klíče mohou být libovolného typu

Výhody Map oproti objektu:

- Klíče mohou být libovolného typu
  - Objekty mohou mít jako klíče pouze string nebo Symbol
- Map ukládá pořadí uložených hodnot
- Map má dobré pomocné funkce

Map, Set, WeakMap, WeakSet

Map je kolekce klíč-hodnota, kde klíče mohou být libovolného typu

Výhody Map oproti objektu:

- Klíče mohou být libovolného typu
  - Objekty mohou mít jako klíče pouze string nebo Symbol
- Map ukládá pořadí uložených hodnot
- Map má dobré pomocné funkce
- Map je škálovatelná a velmi efektivní

Map, Set, WeakMap, WeakSet

```
// Creating a Map
const map = new Map();

// Adding values
map.set("string key", "value");
map.set(42, "number as key");
map.set({}, "object as key");

// Getting values
console.log(map.get("string key")); // value
console.log(map.size); // 3
```

Map, Set, WeakMap, WeakSet

Rozdíly mezi Map a objektem:

```
// Object can only have string/symbol keys
const obj = {
  "1": "string key",
  [Symbol()]: "symbol key"
};
```

Map, Set, WeakMap, WeakSet

```
// Map can have any type of keys
const map = new Map([
  [1, "number key"],
  [{}, "object key"],
  [() => {}, "function key"]
]);
```

Map, Set, WeakMap, WeakSet

Set je kolekce unikátních hodnot

Map, Set, WeakMap, WeakSet

Set je kolekce unikátních hodnot

```
// Creating a Set
const set = new Set([1, 2, 2, 3, 3, 3]);
console.log(set.size); // 3

// Adding and removing values
set.add(4);
set.delete(1);
```

Map, Set, WeakMap, WeakSet

```
// Checking existence
console.log(set.has(2)); // true

// Iteration
for (const value of set) {
  console.log(value);
}
// 2 3 4
```

Map, Set, WeakMap, WeakSet

WeakMap a **WeakSet** jsou slabé verze Map a Set

Map, Set, WeakMap, WeakSet

WeakMap a **WeakSet** jsou slabé verze Map a Set

- Klíče musí být objekty

Map, Set, WeakMap, WeakSet

WeakMap a **WeakSet** jsou slabé verze Map a Set

- Klíče musí být objekty
- Reference na objekty jsou “slabé”

Map, Set, WeakMap, WeakSet

WeakMap a **WeakSet** jsou slabé verze Map a Set

- Klíče musí být objekty
- Reference na objekty jsou "slabé"
  - Garbage collector může objekt uvolnit, pokud na objekt existuje jenom slabá reference

Map, Set, WeakMap, WeakSet

WeakMap a **WeakSet** jsou slabé verze Map a Set

- Klíče musí být objekty
- Reference na objekty jsou "slabé"
  - Garbage collector může objekt uvolnit, pokud na objekt existuje jenom slabá reference
- Nelze iterovat přes hodnoty

Map, Set, WeakMap, WeakSet

WeakMap a **WeakSet** jsou slabé verze Map a Set

- Klíče musí být objekty
- Reference na objekty jsou "slabé"
  - Garbage collector může objekt uvolnit, pokud na objekt existuje jenom slabá reference
- Nelze iterovat přes hodnoty
- Nemají vlastnost size

Map, Set, WeakMap, WeakSet

WeakMap a **WeakSet** jsou slabé verze Map a Set

- Klíče musí být objekty
- Reference na objekty jsou "slabé"
  - Garbage collector může objekt uvolnit, pokud na objekt existuje jenom slabá reference
- Nelze iterovat přes hodnoty
- Nemají vlastnost size

Používá se pro privátní data, caching nebo metadata

Map, Set, WeakMap, WeakSet

```
const weakMap = new WeakMap();
let obj = { data: "some data" };

weakMap.set(obj, "metadata");
console.log(weakMap.get(obj)); // "metadata"

obj = null; // object can be garbage collected
// reference in weakmap will also be removed
```

Map, Set, WeakMap, WeakSet

Použití WeakMap pro privátní data:

```
const privateData = new WeakMap();

class Person {
  constructor(name) {
    privateData.set(this, { name });
  }
  getName() {
    return privateData.get(this).name;
  }
}
```

Map, Set, WeakMap, WeakSet

```
const person = new Person("John");
console.log(person.getName()); // "John"
// privateData is automatically cleaned up when person
// is no longer needed
```

Nové metody pro pole

find a **findIndex** - hledání prvků

```
const users = [
  { id: 1, name: "John" },
  { id: 2, name: "Jane" },
  { id: 3, name: "Bob" }
];
```

Nové metody pro pole

```
// find returns the first matching element
const user = users.find(u => u.id === 2);
console.log(user); // { id: 2, name: "Jane" }

// findIndex returns the index of the first matching element
const index = users.findIndex(u => u.name === "Bob");
console.log(index); // 2
```

Nové metody pro pole

includes - kontrola existence prvku (ES7)

```
const numbers = [1, 2, 3];

// Old way
console.log(numbers.indexOf(2) !== -1); // true

// New way
console.log(numbers.includes(2)); // true
console.log(numbers.includes(4)); // false
```

Nové metody pro pole

flat a **flatMap** (ES10) - zploštění vnořených polí

Nové metody pro pole

flat a **flatMap** (ES10) - zploštění vnořených polí

`flat` zploští vnořená pole volitelné hloubky do pole (základní hloubka je 1)

```
const nested = [1, [2, 3], [4, [5, 6]]];
console.log(nested.flat());    // [1, 2, 3, 4, [5, 6]]
console.log(nested.flat(2));   // [1, 2, 3, 4, 5, 6]
```

Nové metody pro pole

`flatMap` nejdříve transformuje prvky pole a poté je zploští

```
const sentences = ["Hello world", "JavaScript is fun"];
const words = sentences.flatMap(
  s => s.split(" ")
);

console.log(words);
// ["Hello", "world", "JavaScript", "is", "fun"]
```

Nové metody pro objekty

Object.assign - kopírování vlastností objektů

```
const target = { a: 1 };
const source1 = { b: 2 };
const source2 = { c: 3 };

const result = Object.assign(target, source1, source2);
console.log(result); // { a: 1, b: 2, c: 3 }
console.log(target); // { a: 1, b: 2, c: 3 }

// Creating a new object
const clone = Object.assign({}, target);
```

Nové metody pro objekty

Object.fromEntries (ES10) - opak Object.entries

```
const entries = [
  ["name", "John"],
  ["age", 30],
  ["city", "New York"]
];

const obj = Object.fromEntries(entries);
console.log(obj);
// { name: "John", age: 30, city: "New York" }
```

Nové metody pro objekty

```
// Example with Map
const map = new Map(entries);
const objFromMap = Object.fromEntries(map);
```

Shrnutí

Přínos ES6

Co nám ES6 přineslo:

Přínos ES6

Co nám ES6 přineslo:

- **let/const** a blokový scope místo `var`

Přínos ES6

Co nám ES6 přineslo:

- **let/const** a blokový scope místo `var`
- **Arrow funkce**, template literals a destructuring

Přínos ES6

Co nám ES6 přineslo:

- **let/const** a blokový scope místo `var`
- **Arrow funkce**, template literals a destructuring
- **Třídy** a **moduly** – základ moderní architektury

Přínos ES6

Co nám ES6 přineslo:

- **let/const** a blokový scope místo `var`
- **Arrow funkce**, template literals a destructuring
- **Třídy** a **moduly** – základ moderní architektury
- **Promises** – konec callback hell

Přínos ES6

Co nám ES6 přineslo:

- **let/const** a blokový scope místo `var`
- **Arrow funkce**, template literals a destructuring
- **Třídy** a **moduly** – základ moderní architektury
- **Promises** – konec callback hell
- Nové kolekce **Map** a **Set**, iterátory a generátory

Přínos ES6

ES6 byl revoluce – od roku 2015 ale vychází **každý rok** nová verze

Přínos ES6

ES6 byl revoluce – od roku 2015 ale vychází **každý rok** nová verze

Co přinesly verze **ES2016 až ES2024**, si ukážeme v příští přednášce